

Predictive Storage Caching and Tiered Data Migration

How Is This Problem Solved Today?

Classical caching policies (LRU, LFU, ARC)

Learning-assisted caching (LARU, Reuse distance prediction)

RL-based tiered migration

Preprint: Toward Robust and Efficient ML-Based GPU Caching for Modern Inference

Peng Chen¹, Jiaji Zhang¹, Hailiang Zhao^{1*}, Yirong Zhang¹, Jiahong Yu¹, Xueyan Tang², Yixuan Wang³,
Hao Li⁴, Jianping Zou⁴, Gang Xiong⁴, Kingsum Chow¹, Shuibing He¹, Shuiguang Deng^{1*}

¹Zhejiang University ²Nanyang Technological University
³Nanjing University of Aeronautics and Astronautics ⁴Kuaishou

Definition: LARU (Learning-Augmented LRU) is a deterministic caching algorithm designed for GPU inference workloads.

Core Objective: To bridge the gap between simple heuristics (LRU) and machine-learning (ML) based caching.

Primary Goal: Achieve near-optimal performance with accurate predictions while maintaining a safe fallback during prediction errors.

Target Use Cases: DLRM and Large Language Models (LLMs)

The Mechanism

Phase-Based Execution: Operation is divided into phases; a new phase starts when every "old" item in the cache has been either accessed or evicted.

Adaptive Confidence Control: Adjusts a parameter to control the size of the prediction candidate set based on detected errors.

Dynamic Filtering:

- If an error is detected: Fall back to LRU and reduce confidence.
- If no error: Select an eviction candidate from the top-I LRU items using ML predictions

Performance & Deployment Results

DLRM Gains: Improved throughput by up to **24.2%** on the QB-Video dataset.

LLM Gains: Reduced P99 Tail Latency (TTFT) by up to **28.3%** on the Online-QA dataset.

Low Overhead: The LightGBM predictor runs on the CPU, avoiding GPU resource contention.

Harmonia: A Multi-Agent Reinforcement Learning Approach to Data Placement and Migration in Hybrid Storage Systems

Rakesh Nadig[§] Vamanan Arulchelvan[§] Rahul Bera[§] Taha Shahroodi[§] Gagandeep Singh[†]
Andreas Kakolyris[§] Mohammad Sadrosadati[§] Jisung Park[▽] Onur Mutlu[§]
[§]ETH Zürich [†]AMD Research [▽]POSTECH

Why RL?

Harmonia uses RL because traditional storage management often relies on static, human-tuned "heuristics" that cannot keep up with modern data centers. RL provides:

- **Online Adaptability:** It learns directly from live workloads without needing pre-labeled training data.
- **Long-Term Optimization:** Unlike simple rules, RL understands that moving data now might save significant time/latency in the future.
- **Specialization via Multi-Agent RL:** Using two agents prevents "policy interference". A single agent trying to do both tasks often gets confused, but specialized agents can master their specific roles.

How it Works: The RL Formulation

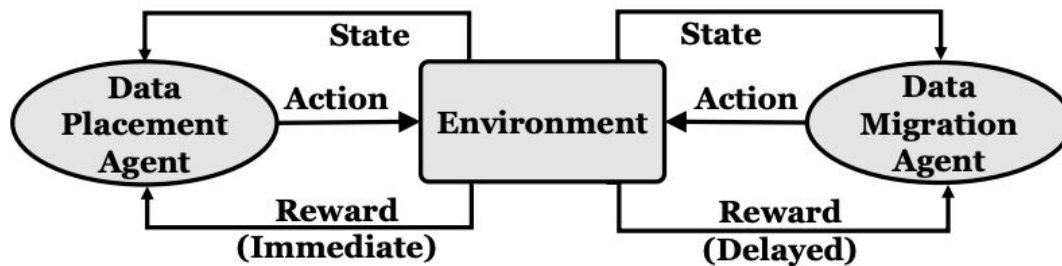


Figure 5: RL Agents in Harmonia.

Why Existing Methods Are Not Enough

Current systems lack:

Integration of prediction + migration

Explicit cost-performance tradeoff modeling

AI workload awareness

Multi-tier optimization across GPU-centric hierarchies

Proposed Framework: Predictive + Adaptive Tier Management

Two Modules:

Predictive Caching Model

- Predict reuse probability
- Estimate time-to-next-access

RL Migration Controller

- Observes tier state
- Decides promotion / demotion

Predictive Model Details

Inputs

- Access history sequence
- Recency
- Frequency
- Object size
- Multi-Tier State

Model Type

- LightGBM

Outputs

$$\hat{P}_i = P(\text{reuse})$$

$$\hat{T}_i = \text{time-to-next-access}$$

RL Formulation

State:

- Tier utilization
- Current placement
- Predicted hotness
- Bandwidth availability

Action:

- Promote object
- Demote object
- Batch migration

Reward:

$$R_t = -(\alpha L_t + \beta C_t + \gamma M_t)$$

Cost–Performance Modeling

In tiered storage systems:

- Fast tiers (HBM, DRAM) → low latency, high cost, small capacity
- Slow tiers (NVMe, SSD, object storage) → high latency, low cost, large capacity

If we only optimize for:

- Latency → everything moves to HBM (impossible due to capacity + cost)
- Cost → everything stays on SSD (terrible performance)
- Migration reduction → no adaptation (high miss rate)

So this is a multi-objective optimization problem.

Latency Model

Per-Object Latency

For each data object i :

$$Latency_i = L_{tier(i)} + AccessPenalty_i$$

Where:

- $tier(i)$ → current storage tier of object i
- $L_{tier(i)}$ → base access latency of that tier
- $AccessPenalty_i$ → contention / transfer overhead

Total System Latency

$$TotalLatency = \sum_i \hat{P}_i \cdot L_{tier(i)}$$

$\hat{P}_i$ = predicted reuse probability

$L_{tier(i)}$ = latency of the tier storing object i

Storage Cost Model and Migration Cost Model

$$StorageCost = \sum_t (DataSize_t \cdot C_t)$$

HBM → expensive

Object storage → cheap

Balances performance vs cost.

$$MigrationCost_i = \frac{Size_i}{Bandwidth} \times PenaltyFactor$$

Prevents excessive thrashing.

$$J = \alpha \cdot TotalLatency + \beta \cdot StorageCost + \gamma \cdot MigrationCost$$

Table 1: Comparison Between Harmonia and Our Framework

Feature	Harmonia	Our Framework
RL-based migration	✓	✓
Explicit reuse prediction	×	✓
Time-to-next-access modeling	×	✓
Predictive signals fed into RL state	×	✓
Multi-tier cost modeling	Partial	Explicit
AI training workload awareness	Not targeted	Designed for it
Thrashing control via foresight	Indirect	Direct

Small exp on dl workflows and existing caching algos on hbm, ssd , nvme, and see what is going at what time